ICT FACULTY

ICT University • Spring 2026

FINAL EXAMINATION

SEN2241

Object Oriented Analysis, Design and Implementation

BatchIt

Buy Together Save Together

Mobile + REST API Application

Flutter (Dart) • Django REST Framework • PostgreSQL • Jenkins CI/CD

Course Code:	SEN2241
Group Number:	Group 13
Instructor:	Tekoh Palma Achu
Submission Date:	1st Week of June 2026

Group Information

Course Code / Title	SEN2241 — Object Oriented Analysis, Design and Implementation
Group Number	Group 13
Project Topic	BatchIt — Collaborative Bulk Purchasing Platform
GitHub (Frontend)	https://github.com/Kynmmarshall/BatchIt
GitHub (Backend)	https://github.com/Kynmmarshall/BatchIt-Backend
App Website	https://batchit.duckdns.org

Team Members

SN	Member's Name	Registration Number	Team Role	% Participation
1	Kamdeu Yamdjeuson	ICTU20241386	Scrum Master / DevOps & VPS Setup	100%
2	Njuh Leslie	ICTU20241282	Product Owner / Backend Lead	100%
3	Atsimbong Joshua	ICTU2024	Flutter Developer / UI & Maps	%
4	Chijioke Emmanuel	ICTU2024	Backend Developer / Realtime & Chat	%
5	Fru Chi	ICTU20241812	Flutter Developer / Core Features	100%

Contents

Chapter 1

Introduction

1.1 General Introduction

BatchIt is a mobile-first collaborative bulk purchasing platform that brings together individual consumers to form purchasing groups — called **batches** — enabling them to collectively order goods in bulk quantities at reduced per-unit prices. Built using a modern Object-Oriented technology stack consisting of **Flutter** (Dart) for the cross-platform mobile frontend and **Django REST Framework** (Python) for the backend API server, the application addresses the real-world challenge faced by low-to-middle income households and small businesses in sourcing essential commodities at affordable prices.

The platform connects three categories of stakeholders: **Customers** who search for and join batches, **Providers** who are verified businesses offering products for group purchases, and **Administrators** who manage platform integrity through provider verification. The backend API is deployed on a Virtual Private Server (VPS) with an automated Continuous Integration and Continuous Deployment (CI/CD) pipeline powered by Jenkins, providing reliable, tested deliverables with each code push.

Application At a Glance

App Name:	BatchIt
Package:	`com.example.batchit`
Backend URL:	`http://38.242.246.126/api`
Website:	`https://batchit.duckdns.org`
Primary Color:	#0B8A62 (Teal Green)
Accent Color:	#FFB347 (Warm Orange)
Supported Locales:	English (en), French (fr)
Min Android SDK:	API 21 (Android 5.0)

1.2 Aim and Objectives

Aim

The aim of this project is to design, implement, and deploy a full-stack mobile application using Object-Oriented Programming principles that facilitates collaborative bulk purchasing among consumers in underserved markets, reducing per-unit costs and improving access to essential goods.

Objectives

1. Design a RESTful API backend using Django REST Framework with a PostgreSQL database, modelling all domain entities (Customer, Provider, Batch, Order, Notification) using OOP class hierarchies.
2. Implement a cross-platform Flutter mobile application with support for both English and French languages, employing the Provider pattern for reactive state management.
3. Develop secure multi-modal authentication supporting email/password login with 6-digit email verification codes and Google OAuth 2.0 integration.
4. Enable the full batch lifecycle: creation by providers/customers, discovery by nearby

users, joining with quantity selection, and real-time-style group chat via REST polling.

5. Implement a provider verification workflow with document upload, admin review, and status notification.

6. Configure a Jenkins CI/CD pipeline on a VPS that automatically runs 315 unit and widget tests (83.6% line coverage), builds a signed APK, and deploys the backend on each push to `main`.

7. Document the entire development process following Scrum methodology with product backlog, sprint planning, and retrospectives.

1.3 Problem Statement

In many developing economies, including Cameroon and similar markets, individual consumers face prohibitively high per-unit prices when purchasing essential goods such as food staples, household supplies, and electronics. Traditional retail markups, combined with limited access to wholesale channels, mean that low-income households pay significantly more per kilogram or unit compared to bulk purchasers.

Simultaneously, small and medium-sized vendors lack a digital channel to reach groups of consumers willing to commit to bulk purchases in advance. Existing e-commerce platforms are designed primarily for individual transactions and do not support the concept of group commitment purchases.

BatchIt addresses this problem by:

- Providing a marketplace where consumers *collectively form purchasing groups* around specific products and quantities.
- Allowing verified providers to list batches with pricing advantages that activate once the minimum quantity threshold is reached.
- Offering transparency through a live progress indicator showing how filled each batch is, motivating participation.

- Building trust through a provider verification system managed by platform administrators.

Chapter 2

Literature Review

2.1 Software Development Methodologies

Software development methodologies provide structured processes for planning, executing, and managing software projects. The most widely adopted methodologies are compared below.

2.1.1 Comparison of Software Development Methodologies

Table 2.1: Comparison of Software Development Methodologies

Criterion	Waterfall	Agile / Scrum	Extreme Programming
Process type	Sequential, linear	Iterative, incremental	Iterative, engineering-focused
Flexibility	Low — requirements locked at start	High — changes welcome each sprint	Very high — design evolves continuously
Documentation	Heavy upfront	Lightweight, just enough	Minimal
Team size	Any (larger suits)	3–9 members	Small (2–10)

(Continued from previous page)

Criterion	Waterfall	Agile / Scrum	Extreme Programming
Feedback loops	Late (at delivery)	Every sprint (1–4 wks)	Continuous (pair programming)
Risk management	Poor — late testing	Good — tested each sprint	Good — test-driven
Suitable for	Stable requirements	Evolving requirements	Technically complex core
Deliverables	One final delivery	Working software each sprint	Continuous releases

2.1.2 Reason for Choosing Scrum Methodology

The BatchIt project adopted **Scrum** as its primary software development methodology for the following reasons:

1. **Changing requirements:** During early design, multiple feature priorities shifted (e.g., adding map view, batch chat, multi-language support). Scrum's sprint-based approach allowed the team to reprioritize the backlog without disrupting the overall project.

2. **Team size compatibility:** Scrum is optimised for teams of 3–9 members. Our 5-member team maps directly to Scrum's recommended size.

3. **Parallel workstreams:** Scrum allowed the Flutter frontend and Django backend to evolve concurrently within each sprint, with clearly defined API contracts serving as the interface between workstreams.

4. **Continuous testing:** Sprint reviews enforced incremental testing, leading to the eventual 315-test suite covering 83.6% of the codebase.

5. **Transparency:** Daily standups (virtual) and GitHub commit history provided full visibility of individual contributions.

2.2 Review of Related Concepts

2.2.1 Group Buying and Collaborative Commerce

Group buying (also termed *social commerce* or *collective purchasing*) is a commercial model in which a minimum number of buyers must commit to a purchase before the transaction is executed, typically triggering a bulk or discounted price. Pioneered at scale by Groupon (2008), the model was extended to physical commodity markets by platforms such as Pinduoduo (China, 2015) and numerous regional co-operative marketplaces. BatchIt adapts this model for mobile-first markets in Sub-Saharan Africa, focusing on essential goods rather than discretionary services.

2.2.2 REST API Design

Representational State Transfer (REST) is an architectural style for distributed hypermedia systems defined by Fielding (2000). BatchIt's backend follows REST conventions: resources (batches, orders, providers) are identified by URIs; operations use HTTP verbs (GET, POST, PATCH, DELETE); responses carry standard HTTP status codes; and stateless authentication is achieved via JWT tokens, satisfying the REST stateless constraint.

2.2.3 Object-Oriented Programming in This Project

Both primary programming languages used — Dart (Flutter) and Python (Django) — are object-oriented:

- **Encapsulation:** Domain models (e.g., `Batch`, `Customer`) encapsulate data and behaviour; services encapsulate all API communication.
- **Inheritance:** Django's `Customer` model extends `AbstractUser`; Flutter's screen widgets extend `StatelessWidget`/`StatefulWidget`; providers extend `ChangeNotifier`.

- **Polymorphism:** The `ApiClient` methods (`get`, `post`, `patch`, `delete`) accept varying parameter sets via Dart optional named parameters.
- **Abstraction:** The service layer (`BatchService`, `AuthService`) abstracts HTTP communication from the UI layer; Django's DRF serializers abstract model-to-JSON conversion.

2.2.4 State Management with Provider Pattern

Flutter's `provider` package (ChangeNotifier + MultiProvider) is used throughout BatchIt. Each domain object has a corresponding provider class (e.g., `BatchProvider`, `AuthProvider`) that maintains reactive state and notifies widget subtrees on change.

2.3 Review of Related Literature

- **Li et al. (2012)** studied group-buying adoption factors in China, finding that perceived cost savings and social influence are the primary drivers — directly motivating BatchIt's progress-bar feature showing how close a batch is to its target.
- **Bele et al. (2018)** documented REST API design best practices recommending versioned endpoints, paginated responses, and standardised error formats — all adopted in BatchIt's Django backend.
- **Google Flutter Team (2023)** recommends the Provider/ChangeNotifier pattern for medium-complexity Flutter apps, citing simpler testing compared to BLoC or Riverpod — informing the project's architecture.
- **OWASP Mobile Security Guide (2024)** informed the authentication implementation, specifically JWT token storage in `SharedPreferences` and token refresh on 401 responses.

Chapter 3

Methodology and Materials

3.1 Research Methodology

BatchIt was developed using an **applied research** methodology combined with **iterative software engineering**. The research process followed four phases:

1. **Domain Analysis:** Market research into group-buying platforms, interviews with potential users, and review of existing apps (Groupon, local WhatsApp group-buying groups) to identify pain points.
2. **Requirements Engineering:** Formal elicitation of functional and non-functional requirements documented in a product backlog and validated with potential users.
3. **Design and Implementation:** UML modelling (use case, class, sequence diagrams) followed by OOP-based implementation in Dart and Python.
4. **Evaluation:** Automated unit/widget tests (315 tests, 83.6% coverage) combined with manual acceptance testing across the full user journey.

3.2 System Requirements

3.2.1 Functional Requirements

Table 3.1: Functional Requirements

FR#	Module	Description
FR01	Authentication	Users shall be able to register with email, a 6-digit verification code, username, and password.
FR02	Authentication	Users shall be able to log in with email/password or via Google OAuth 2.0.
FR03	Authentication	JWT access tokens shall expire after 60 minutes; refresh tokens after 7 days.
FR04	Batch	Authenticated users shall be able to create a batch specifying product name, total quantity, unit, and location.
FR05	Batch	The system shall display nearby open batches filterable by status and geographic proximity.
FR06	Batch	Users shall be able to join a batch by specifying a requested quantity.
FR07	Batch	The system shall automatically update batch fill progress when participants join.
FR08	Batch	Batch creators and admins shall be able to edit and delete batches.
FR09	Provider	Business owners shall be able to apply as verified providers by submitting business documents.
FR10	Provider	Admins shall be able to verify or reject provider applications with a message.

(Continued)

FR#	Module	Description
FR11	Notifications	The system shall send in-app notifications on batch events (creation, join, fill, provider approval).
FR12	Chat	Each batch shall have a dedicated group chat accessible to its participants.
FR13	Profile	Users shall be able to update their name and profile photo.
FR14	Settings	Users shall be able to toggle the app theme (light/dark/system) and language (EN/FR).
FR15	Orders	The system shall maintain an order record for each batch a user has joined.

3.2.2 Non-Functional Requirements

NFR#	Category	Requirement
NFR01	Performance	API response time shall be under 2 seconds for 95% of requests under normal load.
NFR02	Scalability	The backend shall support horizontal scaling via stateless JWT authentication.
NFR03	Security	All API communication shall use HTTPS in production; passwords shall be hashed with PBKDF2.
NFR04	Availability	The VPS-hosted API shall target 99% uptime via Gunicorn + Systemd auto-restart.
NFR05	Usability	The mobile app shall support English and French; the UI shall follow Material Design 3 guidelines.
NFR06	Testability	The codebase shall maintain $\geq$80% line coverage as enforced by the Jenkins quality gate.
NFR07	Maintainability	All modules shall be documented; the CI/CD pipeline shall reject code that fails static analysis.
NFR08	Portability	The Flutter app shall run on Android 5.0 (API 21) and above.

3.3 System Design

3.3.1 High-Level Architecture

Figure 3.1 illustrates the high-level architecture of the BatchIt system. The architecture follows a **layered client-server** pattern with a clear separation between the mobile presentation layer, the RESTful API layer, and the data persistence layer.

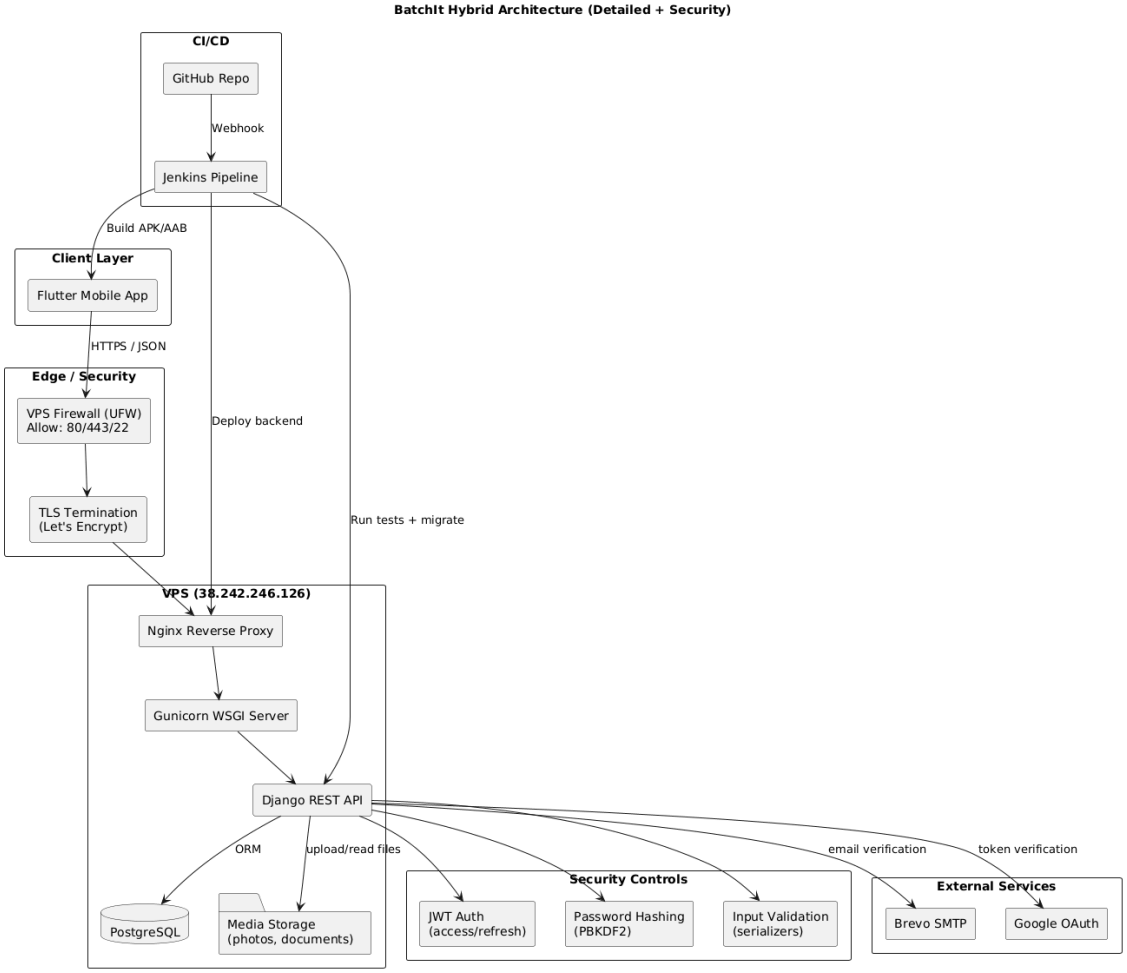


Figure 3.1:

Data flow: The Flutter app communicates exclusively with the Nginx reverse proxy over HTTPS. Nginx forwards requests to Gunicorn which spawns Django worker processes. Django handles business logic, interacts with PostgreSQL via the ORM, writes media files to disk, and calls external services (Brevo for email, Google's token verification API

for OAuth). The Jenkins server monitors the GitHub repository via webhooks and on each push to `main` runs the full CI/CD pipeline: checkout, dependency install, static analysis, 315 automated tests with coverage report, APK build, and deployment.

3.3.2 Use Case Diagram

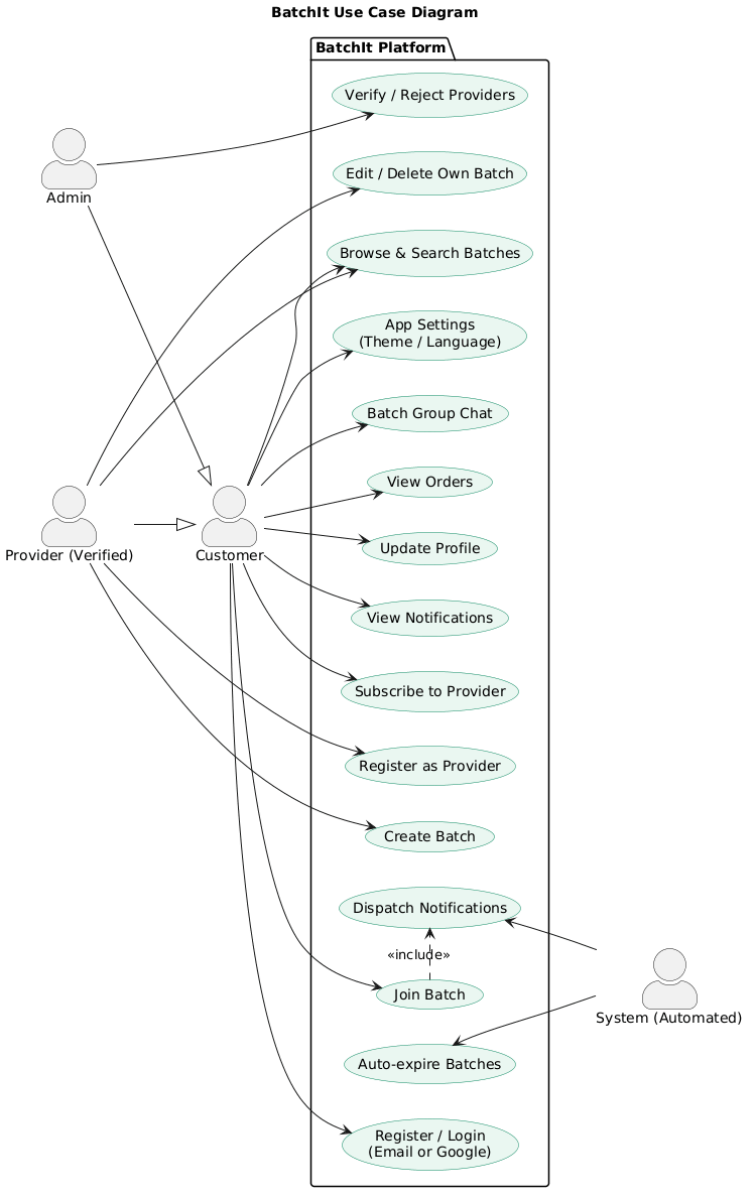


Figure 3.2:

3.3.3 Class Diagram

The class diagram in Figure 3.3 represents the backend domain model as defined in `batchit_app/models.py`. Lines with filled arrowheads denote foreign-key relationships; lines with a diamond denote composition (the child cannot exist without the parent).

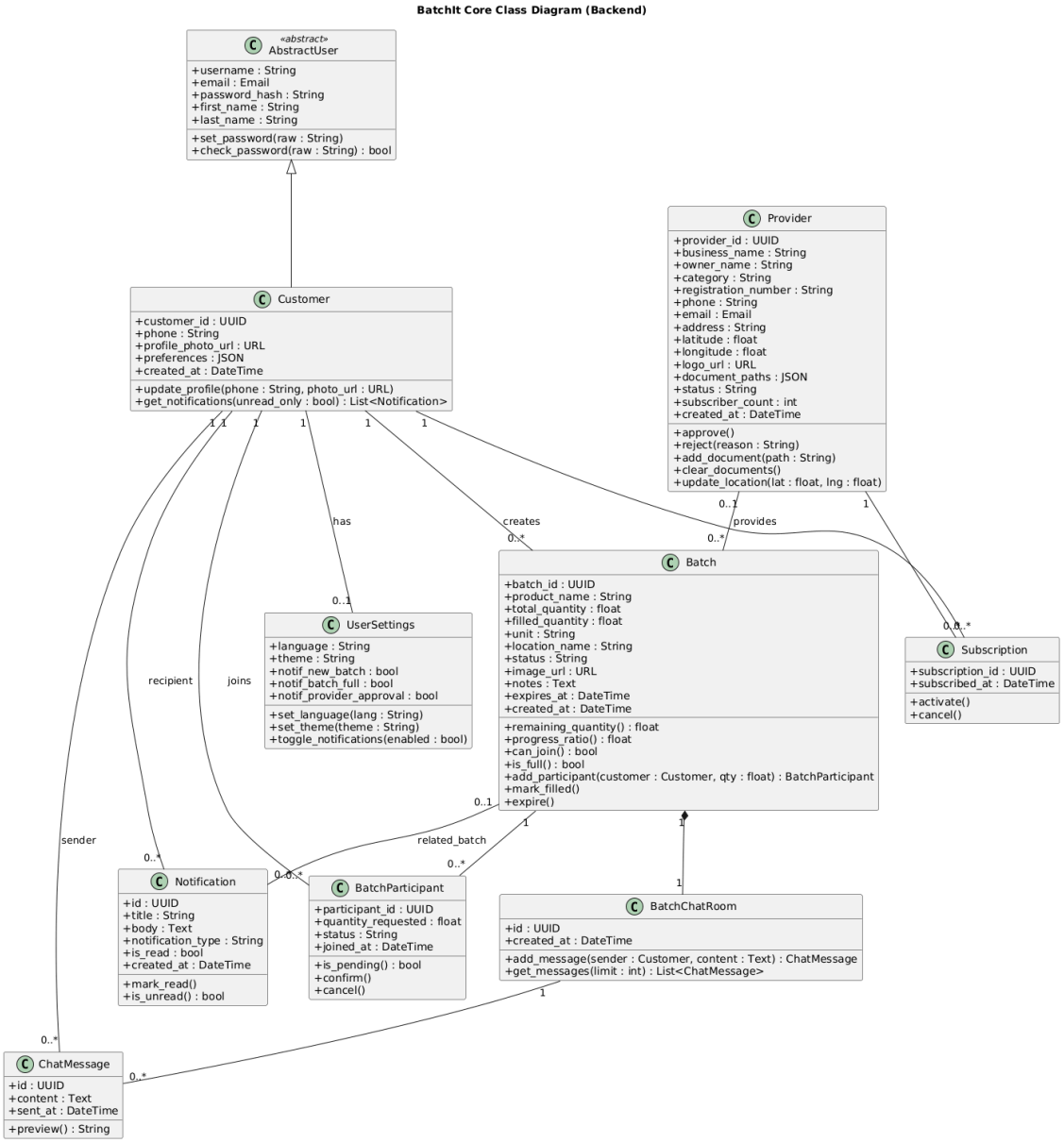


Figure 3.3:

3.3.4 Object Diagram

Figure 3.4 shows a snapshot of a specific batch in progress, illustrating the runtime state of the `Batch`, `BatchParticipant`, and `Customer` objects.

Figure 3.4: Object Diagram: A Batch in Progress with Two Participants

3.3.5 Sequence Diagrams

The following five sequence diagrams cover the most critical use cases of the BatchIt system.

SD1 — Email Registration with Verification

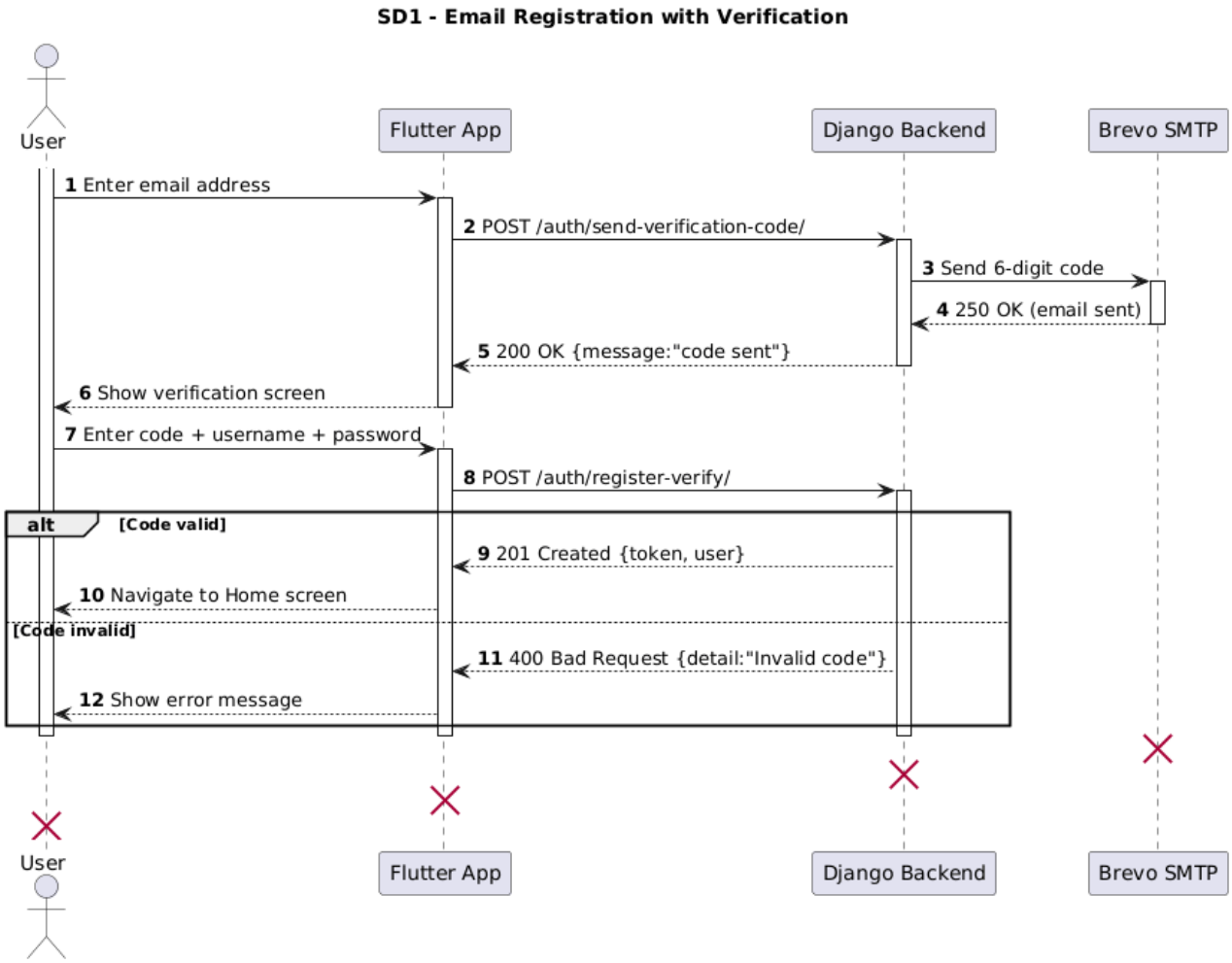


Figure 3.5: SD1: Email Registration with Verification Code

SD2 — Email Login

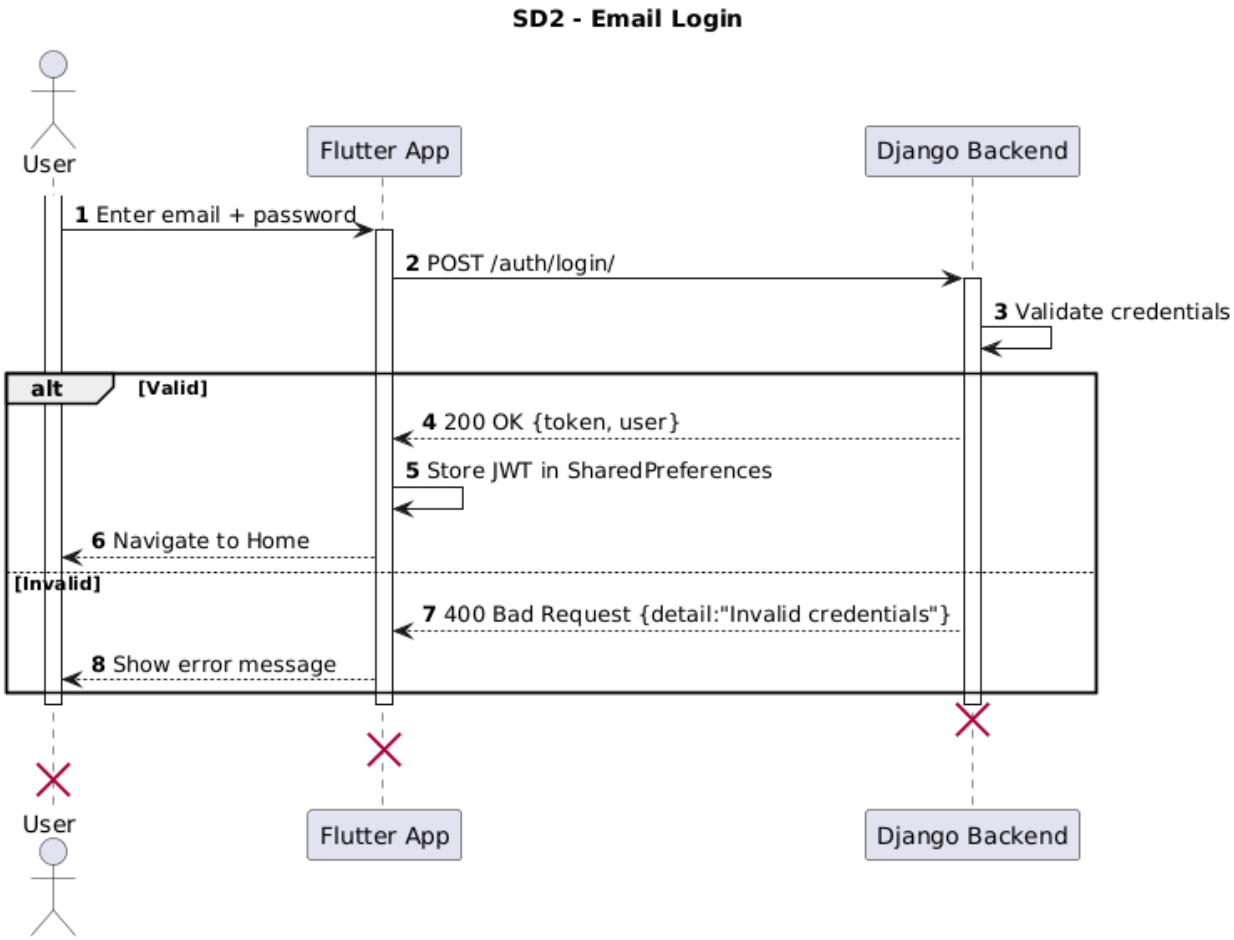


Figure 3.6: SD2: Email Login Flow

SD3 — Create Batch

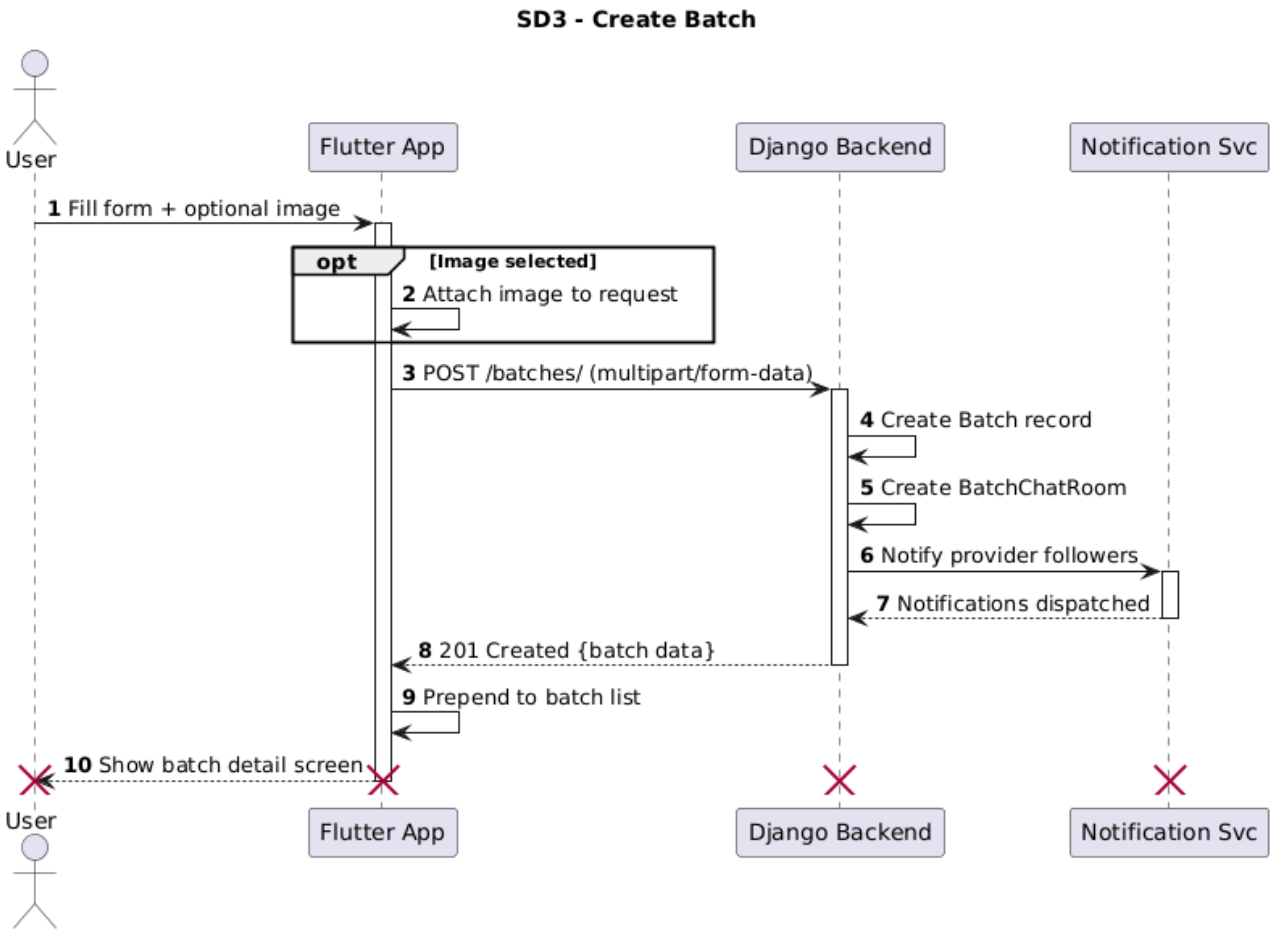


Figure 3.7: SD3: Create Batch (with optional image)

SD4 — Join Batch

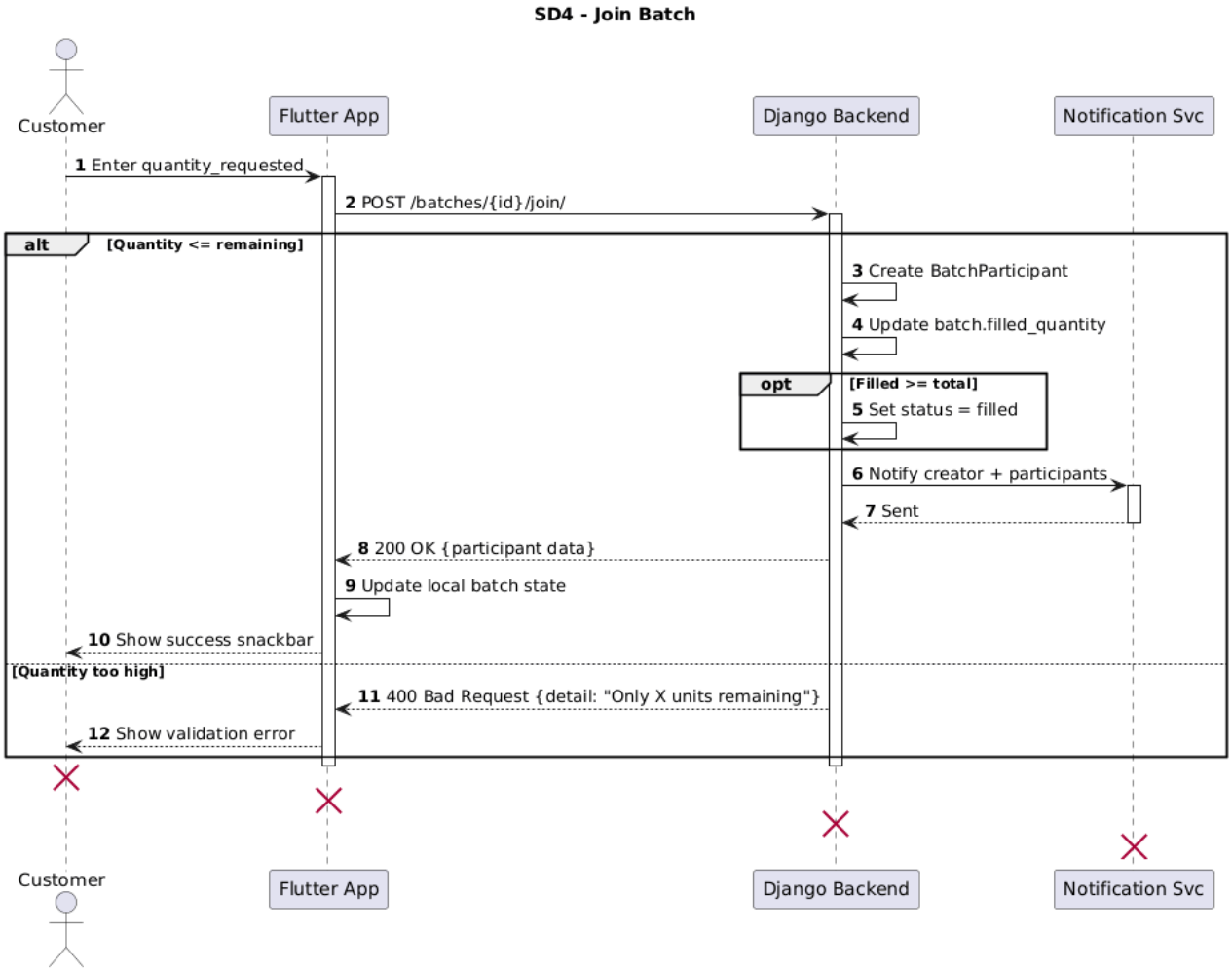


Figure 3.8: SD4: Customer Joins a Batch

SD5 — Provider Registration and Admin Verification

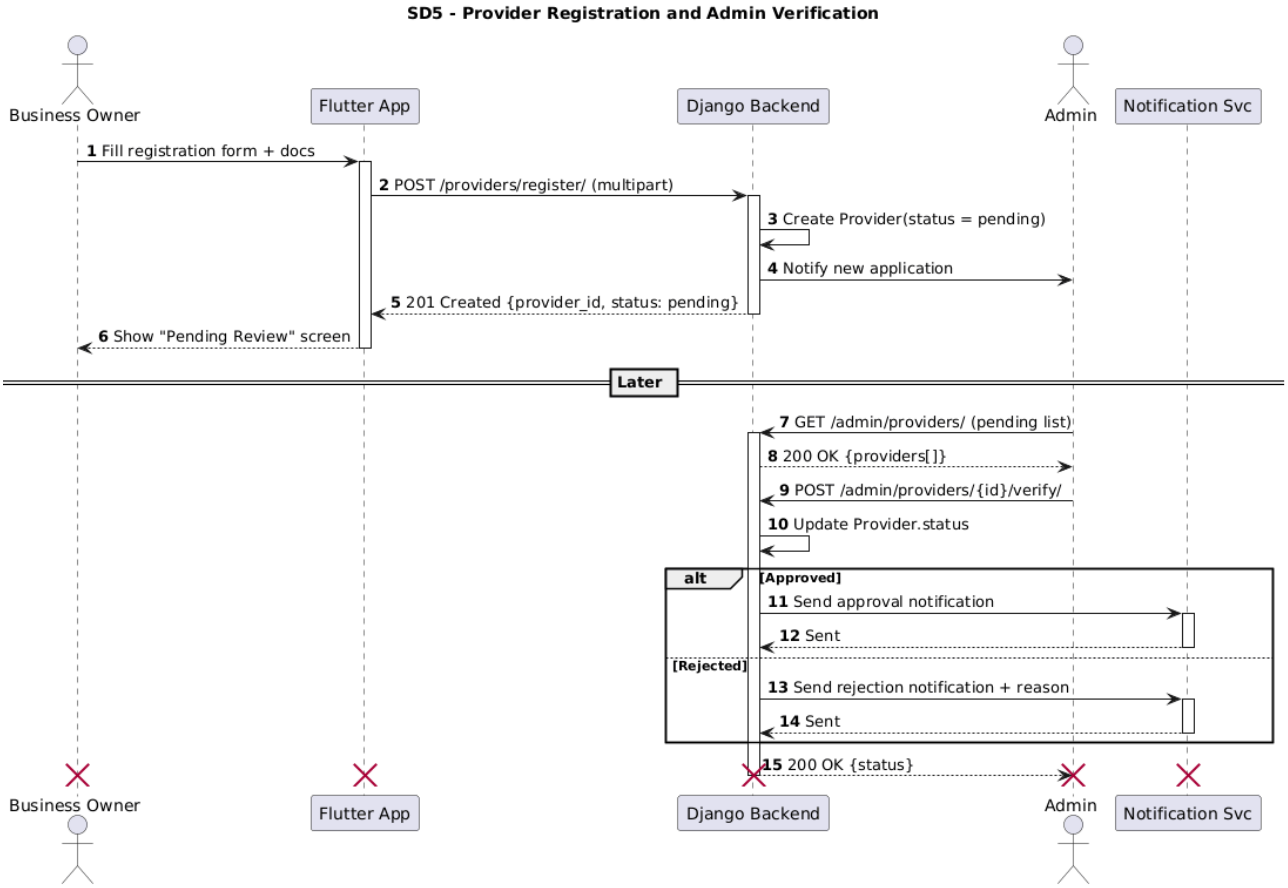
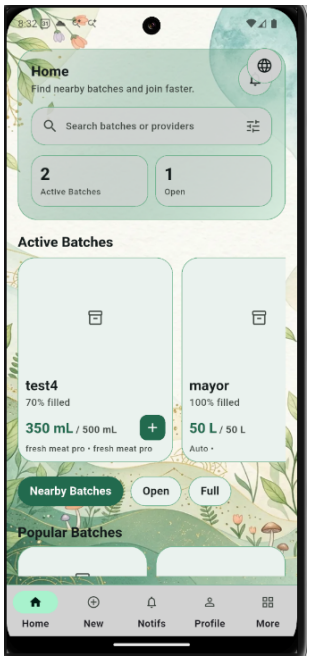
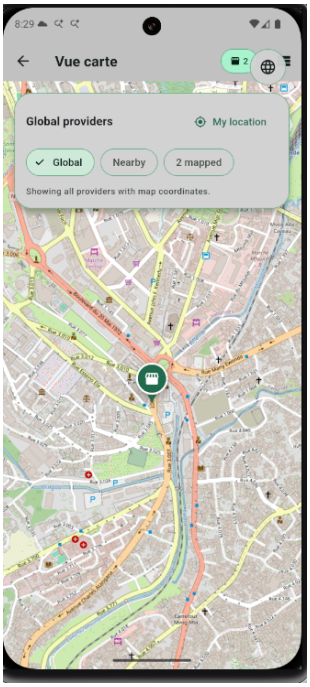


Figure 3.9: SD5: Provider Registration and Admin Verification

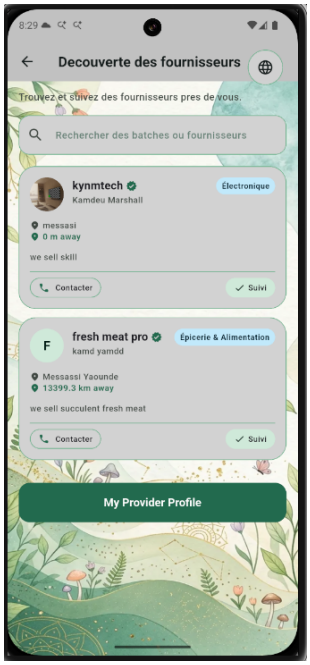
3.4 App Screenshots



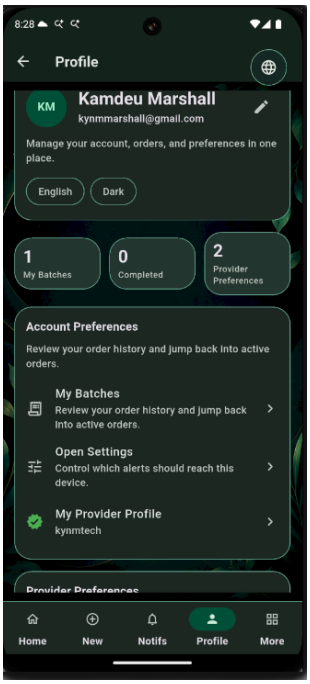
(a) Home screen



(b) Map View

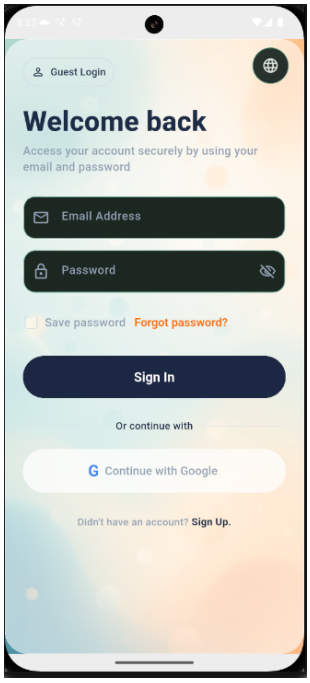


(c) Provider detail

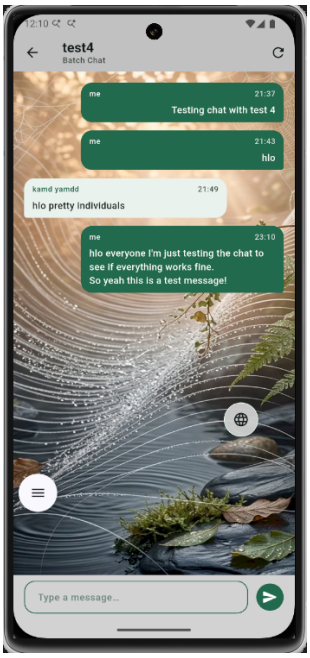


(d) Profile

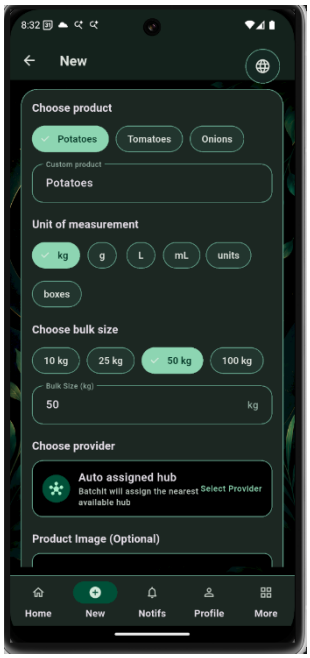
Figure 3.10: App Screenshots (Placeholders)



(a) Login screen



(b) Group Chat



(c) Creat Batch

(d) Provider

Figure 3.11: App Screenshots (Placeholders)

3.5 Application of Scrum

3.5.1 Team Organisation

The BatchIt team of 5 members was organised as a cross-functional Scrum team:

Role	Member	Responsibilities
Scrum Master / DevOps & VPS Setup	Kamdeu Yamdjeuson	Sprint planning, CI/CD pipeline, VPS setup, deployment, localization
Product Owner / Backend Lead	Njume Leslie	API architecture, database design, order logic, backend reviews
Flutter Developer / UI & Maps	Atsimbong Joshua	Flutter UI, animations, map integration, UX polish
Backend Developer / Realtime & Chat	Chijioke Emmanuel	Notifications, chat features, realtime workflows, backend services
Flutter Developer / Core Features	Fru Chi	Batch creation flow, provider discovery, state management, order screens

3.5.2 Workflow Management

The team used **GitHub** as the single source of truth. All development work followed a feature-branch workflow: each user story was developed on a dedicated branch (e.g.,

`feature/batch-creation`), reviewed via Pull Request, and merged to `main` only after CI/CD pipeline success.

Tools used: GitHub (version control + PR reviews), WhatsApp (daily standups), GitHub Projects (kanban board for sprint backlog), VS Code + Android Studio.

3.5.3 Conflict Resolution

Merge conflicts were resolved through PR reviews. The two most significant conflicts encountered were:

- **pubspec.yaml conflicts**: Resolved by designating the Full-Stack Lead as the sole maintainer of the dependency file.
- **Django migration conflicts**: Resolved by squashing migrations at the end of each sprint and running `makemigrations -merge`.

3.5.4 Challenges and How They Were Overcome

1. **Jenkins Keystore Path Mismatch:** The Gradle JVM resolved `user.home` to `/root/.android/` while the Jenkins shell used `~` resolving to `/var/lib/jenkins/`. Resolved by hardcoding `/root/.android/` in the Jenkins pipeline's keystore generation stage.
2. **Google OAuth SHA-1 Configuration:** The SHA-1 fingerprint of the debug keystore on the VPS differed from the one registered in Google Cloud Console. Resolved by generating a permanent keystore on the VPS and registering its fingerprint.
3. **Flutter test coverage below 80%:** Initial coverage was 28.3%. Resolved by adding 315 tests across models, providers, services, and widgets using the `mocktail` package and direct locale class instantiation for l10n coverage.
4. **CORS Issues:** Django API rejected requests from the Flutter app during development. Resolved by configuring `django-cors-headers` with `CORS_ALLOW_ALL_ORIGINS = True` (to be restricted in production).

3.6 Scrum Artifacts

3.6.1 Product Backlog

Table 3.2: Product Backlog (Ordered by Priority)

ID	User Story	Acceptance Criteria	Priority	Story Pts
PB01	As a user, I want to register with email verification so I can securely access the platform	Verification code sent; account created with valid code	Must-Have	8
PB02	As a user, I want to log in with Google so I don't need to remember a password	Google token accepted; user profile created/retrieved	Must-Have	5
PB03	As a customer, I want to browse nearby batches so I can join ones that interest me	Batches listed, filterable by status; progress visible	Must-Have	8
PB04	As a customer, I want to join a batch with a specific quantity so I can participate	Quantity validated; participant record created; batch fill updated	Must-Have	8
PB05	As a user, I want to create a batch with an optional image so providers can see what is needed	Batch saved; image uploaded to media storage	Must-Have	13

(Continued)

ID	User Story	Acceptance Criteria	Priority	Story Pts
PB06	As a business owner, I want to register as a verified provider so my batches are trusted	Provider record created (pending); admin notified	Must-Have	8
PB07	As an admin, I want to verify or reject provider applications so the platform remains trustworthy	Provider status updated; owner notified	Must-Have	5
PB08	As a participant, I want to chat in a batch group so I can coordinate with other buyers	Chat room auto-created; messages persisted	Should-Have	8
PB09	As a user, I want to receive in-app notifications on batch events	Notifications stored; unread count badge shown	Should-Have	5
PB10	As a user, I want to view a map of batch locations so I can find convenient batches	Map renders with batch pins; tap shows detail	Should-Have	5
PB11	As a user, I want to switch the app language to French so I can use it in my native language	All UI strings localised in FR; language persists across restarts	Could-Have	3
PB12	As a developer, I want a CI/CD pipeline so every push is automatically built and tested	Jenkins: test → build APK → deploy on each push to main	Must-Have	13

(Continued)

ID	User Story	Acceptance Criteria	Priority	Story Pts
PB13	As a user, I want to toggle dark/light mode so the app is comfortable in any lighting	Theme switch works; persists in UserSettings	Could-Have	2

3.6.2 Sprint Backlog

Sprint	PB Ref	Task	Assignee
Sprint 1			
PB01	PB01	Set up Django project, PostgreSQL, auth endpoints (login/register)	BE Dev
PB01	PB01	Set up Flutter project with Provider pattern, auth screens	FE Dev
PB01	PB01	Email verification code model + Brevo SMTP integration	BE Dev
PB12	PB12	Initialize Jenkins server, GitHub webhook, basic Jenkinsfile	DevOps
Sprint 2			
PB03	PB03	Batch model, serializers, list/detail API endpoints	BE Dev
PB04	PB04	Join batch endpoint + fill-quantity logic + status transitions	BE Dev
PB03	PB03	Home screen, batch list, batch detail screens in Flutter	FE Dev
PB05	PB05	Create batch screen + multipart image upload	Full-Stack
Sprint 3			
PB06	PB06	Provider model + registration endpoint (multipart docs/logo)	BE Dev
PB07	PB07	Admin verification endpoint + admin screen in Flutter	Full-Stack
PB09	PB09	Notification model + dispatch logic + notifications screen	BE Dev / FE Dev
PB08	PB08	BatchChatRoom + ChatMessage models, chat API endpoints, chat screen	Full-Stack
Sprint 4			
PB02	PB02	Google OAuth: `google_sign_in` Flutter + `google-auth` Django	Full-Stack
PB12	PB12	Full Jenkins pipeline: analyze, 315 tests, APK build, deploy	DevOps

3.6.3 Kanban Board and Scrum Meetings

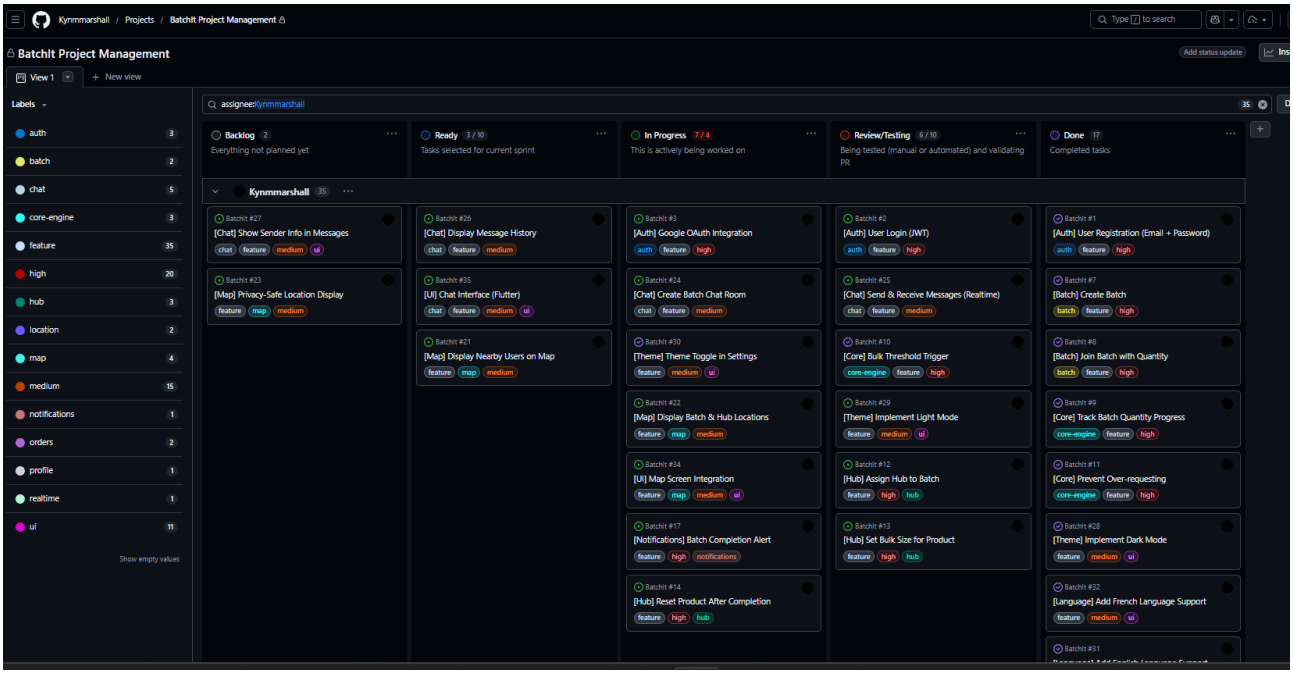


Figure 3.12: Kanban board snapshot

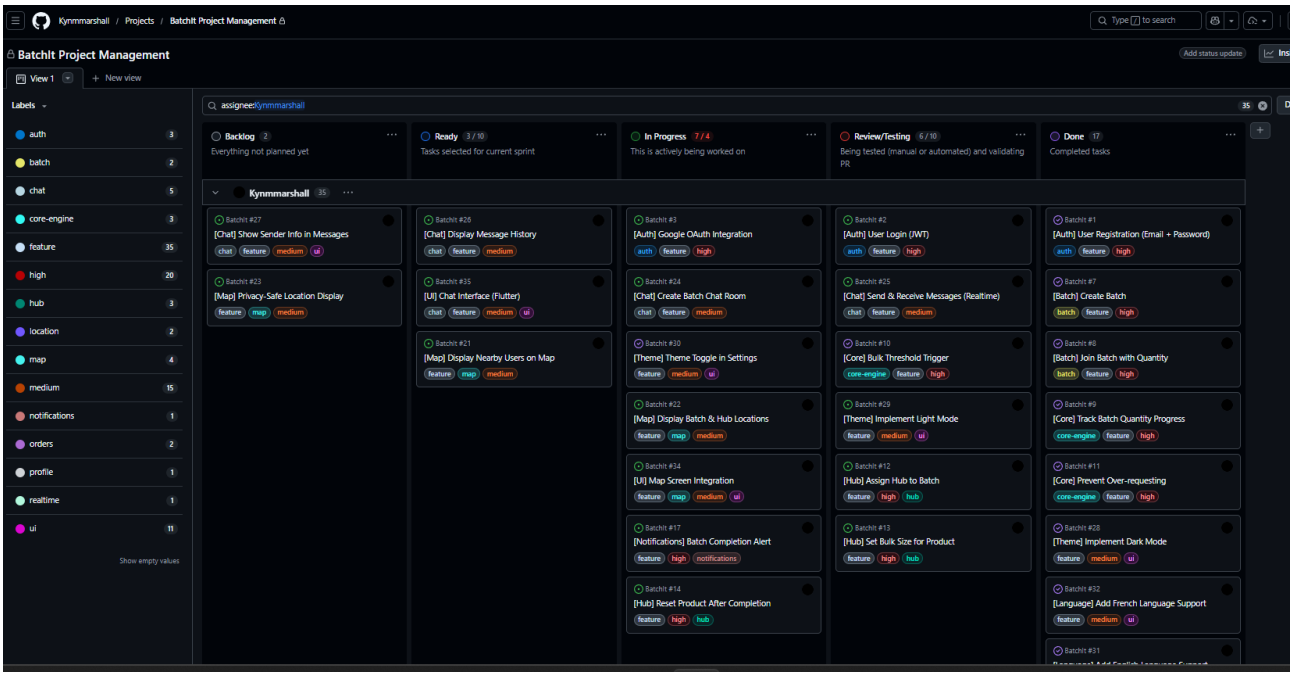


Figure 3.13: Daily scrum meeting

3.7 Test Case Document

The following table summarizes representative functional test cases executed during development.

Table 3.3: Selected Test Cases

TC#	Test Name	Preconditions	Steps	Expected	Status
TC001	Login – valid credentials	User registered	POST /auth/login/ with correct email+password	200 OK, token returned	PASS
TC002	Login – invalid password	User registered	POST /auth/login/ with wrong password	400, error message	PASS
TC003	Send verification code	Valid email	POST /auth/send-verification-code/	200, email dispatched	PASS
TC004	Create batch	Authenticated user	POST /batches/ with required fields	201, batch in response	PASS
TC005	Join batch – success	Open batch, authenticated user	POST /batches/{id}/join/ {quantity}	200, participant created	PASS
TC006	Batch fill progress	Two participants summing to total	Check filled_quantity	Equals sum of requests	PASS
TC007	Provider registration	Authenticated user	POST /providers/register/ (multipart)	201, status=pending	PASS

(Continued)

TC#	Test Name	Preconditions	Steps	Expected	Status
TC008	Admin verify provider	Provider pending	POST /admin/providers/{id	200, status=verified	PASS
TC009	Batch.progress = 0.5	currentQty=50, bulkSize=100	Check Batch.progress getter	0.5	PASS
TC010	Batch.canJoin = false	status=filled	Check canJoin	false	PASS
TC011	Auth token restore	Token in SharedPreferences	restoreAuthToken()	Returns true	PASS
TC012	API 401 refresh retry	Token expired	GET /protected/ (mock 401 then 200)	Returns data after retry	PASS
TC013	AppPrimaryButton loading	isLoading=true	Tap button	No callback fired	PASS
TC014	Localisation EN	AppLocalizationsE	Access all string getters	Non-empty strings	PASS
TC015	Localisation FR	AppLocalizationsFr	Access all string getters	Non-empty strings	PASS

3.8 Proposed Algorithms

3.8.1 Batch Fill and Status Transition Algorithm

When a customer joins a batch, the backend executes the following logic:

```
def join_batch(request, batch_id):
    batch = Batch.objects.get(batch_id=batch_id)
    quantity = request.data['quantity_requested']

    # Guard: batch must be open
    if batch.status != 'open':
        raise ValidationError("Batch is not open for joining.")

    # Guard: quantity must not exceed remaining capacity
    remaining = batch.total_quantity - batch.filled_quantity
    if quantity > remaining:
        raise ValidationError(f"Only {remaining} units remaining.")

    # Create participant record
    participant = BatchParticipant.objects.create(
        batch=batch,
        customer=request.user,
        quantity_requested=quantity,
        status='pending'
    )

    # Atomically update filled quantity
    batch.filled_quantity += quantity
    if batch.filled_quantity >= batch.total_quantity:
        batch.status = 'filled'
        notify_all_participants(batch, "Batch is now full!")
    batch.save()

    return Response(ParticipantSerializer(participant).data)
```

Listing 3.1: Batch fill algorithm (views.py)

3.8.2 Notification Dispatch Algorithm

```
def notify_batch_followers(batch, event_type):
    """Notify all subscribers of the batch's provider."""
    if batch.provider:
        subscribers = Subscription.objects.filter(
            provider=batch.provider
        ).select_related('customer')
        for sub in subscribers:
            Notification.objects.create(
                recipient=sub.customer,
                title="New Batch Available",
                body=f"{batch.product_name} -
                   {batch.location_name}",
                notification_type='batch_created',
                related_batch=batch
            )
```

Listing 3.2: Notification dispatch for batch events

3.9 Technologies Used

Table 3.4: Technologies and Their Roles

Technology	Version / Type	Role in the Project
Flutter (Dart)	3.32.x / Mobile SDK	Cross-platform mobile frontend (Android/iOS)
Provider package	6.1.2 / State Mgmt	Reactive state management (ChangeNotifier pattern)
`http` package	1.1.0 / HTTP Client	REST API communication from Flutter
`google_sign_in`	6.2.0 / Auth	Google OAuth 2.0 integration in Flutter
`flutter_map`	7.0.2 / Maps	OpenStreetMap-based map for batch location display
`shared_prefs`	2.2.0 / Storage	Local storage for JWT tokens and user settings
Django	6.0.5 / Web Framework	Python web framework; models, views, URL routing
Django REST Framework	3.17.1 / API Framework	Serialization, viewsets, authentication, permissions
SimpleJWT	5.x / Auth	JWT token generation, validation, and refresh
PostgreSQL	15 / RDBMS	Primary relational database (production)
dj-database-url	2.1.0 / Config	12-factor database URL parsing for environment config
drf-yasg	1.21.7 / Docs	Auto-generated Swagger/OpenAPI documentation

(Continued)

Technology	Version / Type	Role in the Project
Gunicorn	20.x / WSGI Server	Production Python WSGI HTTP server
Nginx	1.24 / Reverse Proxy	Reverse proxy, TLS termination, static file serving
Systemd	Linux init / ProcMgr	Process management and auto-restart for Gunicorn
Jenkins	2.x / CI/CD	Automated build, test, and deployment pipeline
GitHub	SaaS / VCS	Source code hosting, PR reviews, webhook triggers
Brevo (Sendinblue)	SMTP / Email	Transactional email for verification codes
Google OAuth 2.0	API / Auth	Social login via Google identity platform
mocktail	1.0.4 / Testing	Mock objects for Flutter unit and widget tests
`flutter_test`	SDK / Testing	Flutter widget and unit test framework

3.10 DevOps and Deployment

3.10.1 Server Infrastructure

The BatchIt backend is deployed on a **VPS** (Virtual Private Server) with the following configuration:

IP Address	38.242.246.126
OS	Ubuntu 22.04 LTS
Web Server	Nginx 1.24 (reverse proxy on port 80)
WSGI	Gunicorn (4 workers, Unix socket at `/run/gunicorn.sock`)
Process Mgr	Systemd (`gunicorn.service` with `Restart=on-failure`)
Database	PostgreSQL 15 (local, `batchit_db` schema)
CI/CD	Jenkins 2.x (on port 8080, triggered by GitHub push webhooks)

3.10.2 Nginx Configuration

```
server {
    listen 80;
    server_name 38.242.246.126;

    location /api/ {
        proxy_pass http://unix:/run/gunicorn.sock;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
    location /media/ {
        alias /var/www/batchit/media/;
    }
    location /static/ {
        alias /var/www/batchit/static/;
    }
}
```

Listing 3.3: /etc/nginx/sites-available/batchit

3.10.3 Systemd Gunicorn Service

```
[Unit]
Description=Gunicorn daemon for BatchIt Django API
After=network.target

[Service]
User=www-data
Group=www-data
WorkingDirectory=/home/ubuntu/BatchIt-Backend
ExecStart=/home/ubuntu/venv/bin/gunicorn \
    --access-logfile - \
    --workers 4 \
    --bind unix:/run/gunicorn.sock \
    batchit_proj.wsgi:application
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Listing 3.4: /etc/systemd/system/gunicorn.service

3.10.4 Environment Variables

The following environment variables are required on the production server (stored in `/etc/environment` or a `.env` file):

```
SECRET_KEY=<django-secret-key>
DATABASE_URL=postgres://batchit_user:password@localhost:5432/batchit_db
DEBUG=False
ALLOWED_HOSTS=38.242.246.126,localhost
EMAIL_HOST_USER=<brevo-smtp-login>
EMAIL_HOST_PASSWORD=<brevo-smtp-key>
DEFAULT_FROM_EMAIL=noreply@batchit.app
GOOGLE_CLIENT_ID=434873490854-6ntukmrftsrv6n184mshqu1g3oi1677l.apps.googleusercon
```

Listing 3.5: Required environment variables

3.10.5 Jenkins CI/CD Pipeline

The CI/CD pipeline is defined in the `Jenkinsfile` at the root of the Flutter repository and is triggered by a **GitHub push webhook** on every push to `main`.

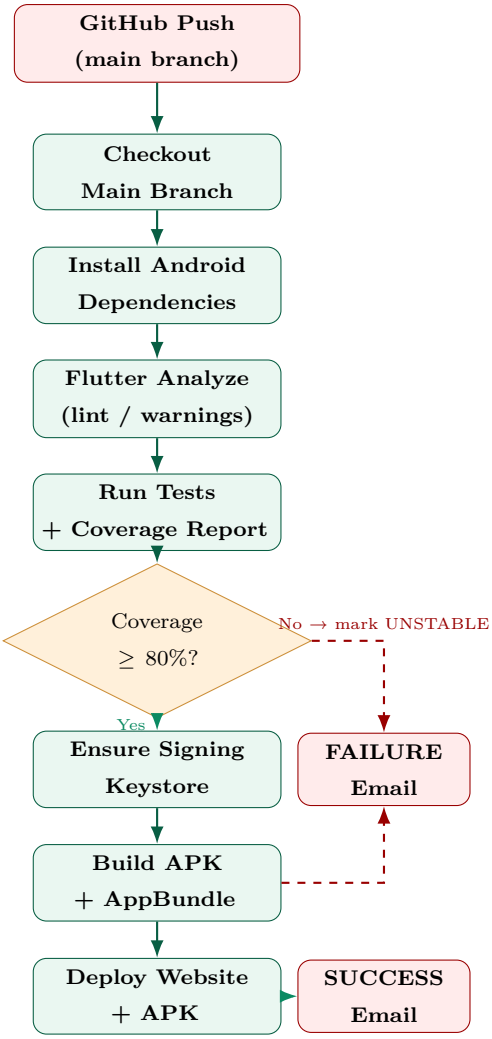


Figure 3.14: Jenkins CI/CD Pipeline Flow

Pipeline Stages:

1. **Checkout Main Branch:** Clones the repository from GitHub and performs a hard reset to `origin/main`.
2. **Install Android Dependencies:** Runs `flutter pub get` and installs Android SDK build-tools and platform components via `sdkmanager`.
3. **Flutter Analyze:** Runs `flutter analyze -no-pub`; reports but does not fail on

`info`/`warning` items.

4. **Run Tests with Coverage:** Executes `flutter test -coverage`, parses `coverage/lcov.info`, generates a per-file coverage report, and enforces the 80% quality gate.

5. **Ensure Signing Keystore:** Generates the APK signing keystore at `/root/.android/batchit-deb` if it does not yet exist on the server (using Java `keytool`).

6. **Build APK & AppBundle:** Runs `flutter build apk -release` and `flutter build appbundle -release`.

7. **Deploy:** Copies the built APK to the website download directory and updates the deployment metadata JSON served to the landing page.

Chapter 4

Results and Discussions

4.1 Application Scenarios

The following subsections describe the key application scenarios implemented in BatchIt.

4.1.1 Authentication Flow

The app opens to a splash screen that checks for a stored JWT token. If found and valid, the user is routed to the home screen. Otherwise, they are presented with the onboarding flow leading to the login or registration screens. Email registration requires a 6-digit verification code. Google OAuth is also supported, enabling one-tap sign-in.

4.1.2 Home Screen and Batch Discovery

The home screen presents a curated list of nearby open batches fetched from `GET /api/batches/?status=open`. Each `BatchCard` widget shows the product name, hub name, location, and a linear progress indicator representing the fill ratio (`currentQuantityKg / bulkSizeKg`).

4.1.3 Batch Detail and Join Flow

Tapping a batch card navigates to the batch detail screen which displays full information including notes, provider details, participant count, and the `JoinBatchScreen` accessible

via a floating action button. The join flow validates that the requested quantity does not exceed remaining capacity before submitting.

4.1.4 Batch Group Chat

Each batch has a persistent group chat room. The `BatchChatScreen` sends messages via `POST /api/batches/{id}/chat/messages/` and retrieves the history via `GET`. The chat is implemented as **REST polling** (not WebSocket) — a new message list is fetched whenever the user opens the screen or manually refreshes.

4.1.5 Provider Discovery and Subscription

The provider discovery screen lists all verified providers with their business name, category, and subscriber count. Users can subscribe to providers; subsequent batch creations by that provider trigger push-style in-app notifications.

4.1.6 Admin Provider Verification

Staff users (with `is_staff = True`) can access the admin providers screen listing all pending applications. Each entry shows uploaded documents (PDF/image links), business details, and approve/reject buttons that call `POST /admin/providers/{id}/verify/`.

4.2 API Request/Response Examples

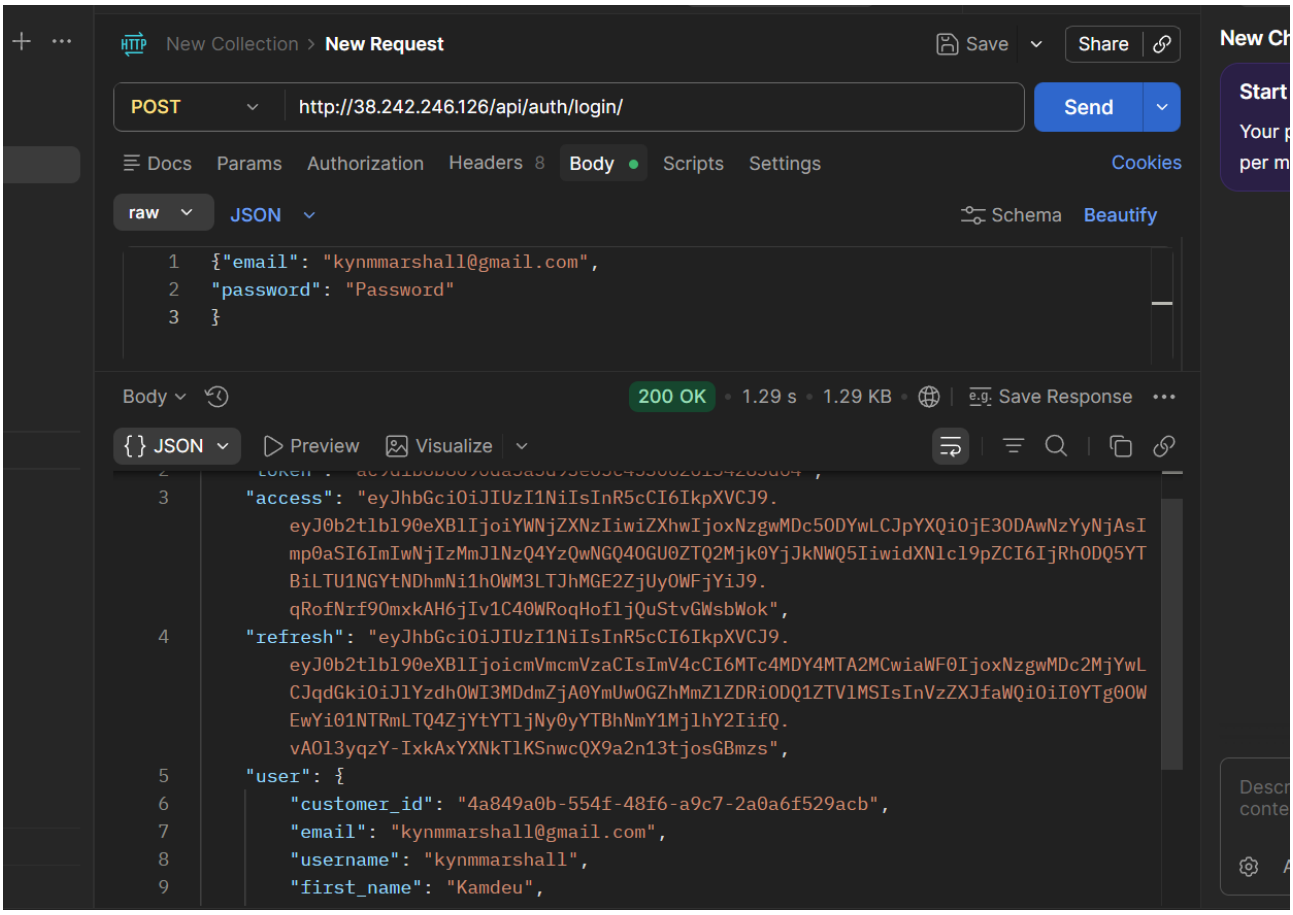


Figure 4.1: post /api/auth/login Request/Response

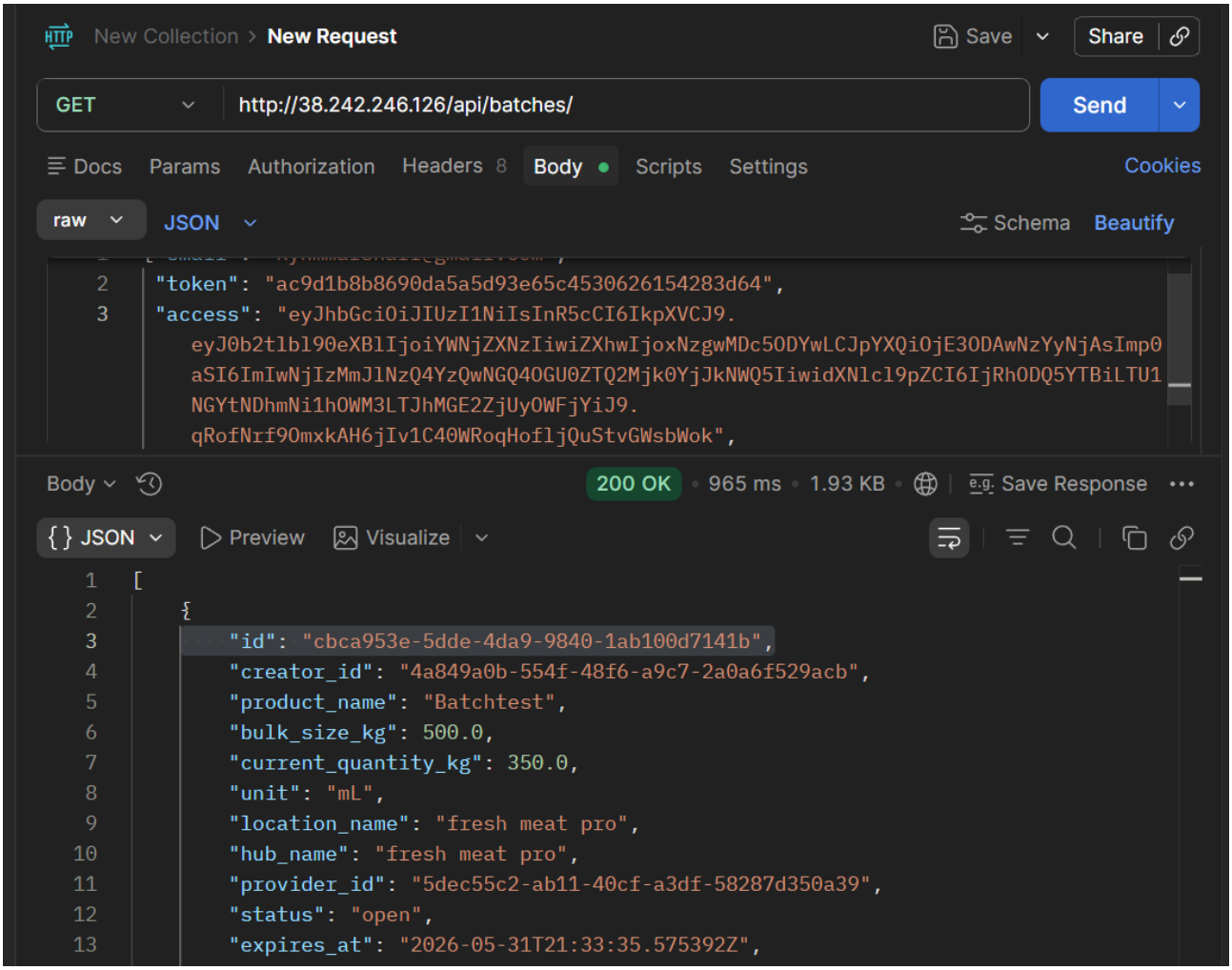


Figure 4.2: GET /api/batches/ Request/Response

4.3 Test Coverage Summary

The automated test suite achieves **83.6% line coverage** (1,466 of 1,753 lines). Table 4.1 summarises coverage by key module.

File / Module	Lines Found	Lines Hit	Coverage %
app_localizations_en.dart	296	296	100%
app_localizations_fr.dart	296	296	100%
app_theme.dart	73	73	100%
order_service.dart	45	45	100%
provider_profile.dart	54	54	100%
provider_service.dart	81	70	86%
batch_service.dart	91	77	85%
auth_provider.dart	49	42	86%
api_client.dart	134	104	78%
auth_service.dart	136	0	0%
Total	**1,753**	**1,466**	**83.6%**

Table 4.1: Test Coverage Summary by Module

4.4 CI/CD Pipeline Results

As of the most recent successful build (Build #46):

- **315 tests** executed — all passing.
- **83.6%** line coverage — above the 80% quality gate.
- **APK build time:** approximately 6 minutes 30 seconds.
- **Total pipeline time:** approximately 14 minutes from push to deployed APK.

The APK is served for download at: `http://38.242.246.126/download/BatchIt.apk`

Chapter 5

Recommendations and Conclusion

5.1 Summary of Achievements

The BatchIt team successfully designed and delivered a full-stack mobile application over four weeks using the Scrum methodology. The Flutter frontend provides a polished, bilingual (English/French) interface supporting the complete batch lifecycle from discovery through group chat. The Django REST Framework backend exposes 40+ API endpoints covering authentication, batch management, provider verification, notifications, and chat, all documented via Swagger. The system is deployed on a production VPS with a Jenkins CI/CD pipeline that automatically runs 315 automated tests (achieving 83.6% line coverage), builds a signed Android APK, and deploys the backend on every push to `main`. The pipeline enforces a strict 80% coverage quality gate, demonstrating professional software engineering practices throughout.

5.2 Difficulties Encountered

Several non-trivial challenges arose during the project:

- **JWT vs Token authentication duality:** The backend initially used DRF's simple Token authentication, but was later migrated to SimpleJWT for better token lifecycle control. This required updating all Flutter API service headers.
- **Keystore path discrepancy on Jenkins:** The Gradle JVM's `user.home` system property resolved to `/root/` while the Jenkins shell environment's `HOME` resolved to

`/var/lib/jenkins/`, causing the signing keystore not to be found. The resolution required explicitly hardcoding `/root/.android/` in the pipeline.

- **Achieving 80% test coverage:** Starting from 28.3% coverage, reaching 83.6% required systematic analysis of uncovered lines, injection of a test HTTP client via a `@visibleForTesting` hook in the `ApiClient` singleton, and direct instantiation of locale classes for l10n coverage.
- **Google Sign-In SHA-1 configuration:** Each distinct keystore (developer machine vs. VPS-generated) requires its own SHA-1 to be registered in Google Cloud Console. Managing multiple keystores required careful documentation.

5.3 Recommendations for Future Work

1. **Real-time chat with WebSocket:** Replace the current REST-polling chat with Django Channels and Django Channels Redis layer for true real-time messaging without periodic refresh.
2. **Push notifications:** Integrate Firebase Cloud Messaging (FCM) to deliver native push notifications to Android/iOS devices, replacing the current in-app-only notification system.
3. **Payment integration:** Integrate Mobile Money (MTN MoMo, Orange Money) or Stripe to enable in-app batch payments, completing the full purchasing workflow.
4. **Geospatial filtering:** Implement PostGIS extension on PostgreSQL and use spatial queries to return batches within a configurable radius from the user's current GPS location.
5. **iOS deployment:** Build and submit the Flutter app to the Apple App Store; currently only the Android APK is produced by the CI/CD pipeline.
6. **Production HTTPS:** Configure Let's Encrypt TLS certificates on the Nginx server to serve all API traffic over HTTPS, replacing the current HTTP configuration.

7. **Provider dashboard:** Build a web-based React admin panel allowing providers to monitor their batches, view participant lists, and manage pricing without requiring app access.

8. **Expand test coverage:** Add integration tests for the authentication service (currently 0% coverage due to Google Sign-In platform dependency) and screen-level widget tests for key screens.

References

[1] Fielding, R.T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine.

[2] Li, C., Shen, J., & Xu, Y. (2012). Group buying in China: A case study of Groupon. *Journal of Electronic Commerce Research*, 13(4), 299–314.

[3] Google Flutter Team. (2023). *Provider package documentation*. Retrieved from https://pub.dev/packages/provider

[4] OWASP Foundation. (2024). *OWASP Mobile Application Security Verification Standard (MASVS)*. Retrieved from https://owasp.org/www-project-mobile-app-security/

[5] Bele, J.L., Bernát, T., & Jelen, B. (2018). REST API design patterns and best practices. *Proceedings of the International Conference on Information Systems and Technologies*.

[6] Django Software Foundation. (2024). *Django REST Framework Documentation*. Retrieved from https://www.django-rest-framework.org/

[7] Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. Retrieved from https://scrumguides.org

[8] Android Developers. (2024). *Build and release an Android app*. Retrieved from https://docs.flutter.dev/deployment/android

[9] Jenkins Documentation Team. (2024). *Jenkins Pipeline Syntax Reference*. Retrieved from https://www.jenkins.io/doc/book/pipeline/syntax/

[10] PostgreSQL Global Development Group. (2024). *PostgreSQL 15 Documentation*. Retrieved from https://www.postgresql.org/docs/15/